

Legal Rights Navigator

Complete Architecture & Design Reference

GenAI Portfolio Project	FastAPI + LangGraph + PGVector	React Frontend
-------------------------	--------------------------------	----------------

An AI-powered platform providing Indian citizens with guided legal assistance across five domains, using a multi-node LangGraph pipeline, hybrid vector search, cost-optimised model routing, and persistent per-user conversation memory.

1. Technology Stack

Layer	Choice	Why
Frontend	React	Component model fits multi-domain chat UI
Backend	FastAPI (Python)	Native async/await; AI libs are Python-first
AI Orchestration	LangGraph	Stateful multi-node agent with conditional edges
Database	PostgreSQL + PGVector	Single DB for SQL + vectors + metadata filtering
Auth	Google OAuth 2.0 + JWT	HttpOnly cookies; no raw tokens on client
Embeddings	text-embedding-3-small	Cost-effective, high quality
Classifier	Claude Haiku	Cheap; domain routing only
Reasoner	Claude Sonnet	Complex legal reasoning + structured JSON output

FastAPI over Express: Flask is synchronous — 5 concurrent AI calls means users 2-5 wait in queue. FastAPI is natively async; it handles thousands of concurrent AI API calls on a single worker. AI libraries (LangChain, LangGraph) are Python-first — no fragmented JS/Python two-server setup needed.

2. The Five Legal Domains

Each domain is fully isolated — its own vector collection, system prompt, document checklist, and template generator.

#	Domain	Key Acts Indexed
1	Labour & Employment	Industrial Disputes Act, Payment of Wages Act, PF Act
2	Consumer Protection	Consumer Protection Act 2019, BIS standards
3	Tenant & Property	Rent Control Acts (state-wise), Transfer of Property Act
4	Cyber Crime & Online Fraud	IT Act 2000, IPC 420/66C/66D
5	Family & Women's Rights	PWDVA, Hindu Marriage Act, DV Act

3. Request Pipeline (LangGraph Nodes)

Node 0	3-Tier Validity Gate	Tier 1: Regex keyword blocklist (<1ms). Tier 2: Embedding cosine similarity vs legal anchor set (~5ms, threshold 0.65). Tier 3 (5% of queries): Haiku binary classifier. Invalid queries are rejected and NEVER written to DB.
Node 1	Domain Classifier	Haiku LLM classifies query into one of 5 domains. Returns {domain, confidence}. If confidence < 0.7, escalates to clarification node before proceeding.
Node 2	Clarification (conditional)	Extracts known facts from LangGraph state. Identifies missing_information. Asks user ONE targeted question. Hard cap: 2 clarification rounds max — avoids infinite loops.
Node 3	Hybrid Retriever	SQL: filter by domain metadata FIRST, then semantic search. Fetch top-20. Cross-encoder reranker narrows to top-3. Prevents labour law chunks surfacing for cybercrime queries.
Node 4	Legal Planner	Receives retrieved chunks + facts + intent signals. Decides output type: EXPLAIN / DRAFT_COMPLAINT / DRAFT_NOTICE / CHECKLIST / REFERRAL. Returns structured JSON plan.
Node 5	Generator (parallel branches)	Advice Generator: prose legal explanation with source citations. Complaint Generator: Jinja2 template filled with structured facts from planner. Both merge into Final Response.

4. LangGraph State Schema

Every node reads from and writes to this shared typed state dict:

```
{
  # Identity
  "session_id":      str,    # conversation thread ID
  "user_id":         str,    # authenticated user UUID
  "turn_number":     int,    # for memory windowing

  # Query analysis
  "user_query":      str,
  "is_valid":        bool,   # passed validity gate?
  "validation_tier": int,    # which tier (1/2/3) processed it
  "domain":          str,    # one of 5 domains
  "confidence":       float,  # classifier confidence

  # Facts & gaps
  "extracted_facts": dict,   # persistent legal facts (name, date, amount...)
  "missing_information": list,
  "clarification_count": int, # hard cap at 2

  # Retrieval
  "retrieved_documents": list, # top-3 reranked chunks

  # Planning
  "reasoning":        str,
  "plan":             dict,   # {type, action, template_vars}

  # Output
  "response":         str,
  "error_state":       str | None
}
```

5. Vector DB & Ingestion Pipeline

Why PGVector over Qdrant/Chroma/FAISS?

DB	Verdict	Reason
PGVector ■	CHOSEN	Single DB for SQL + vectors. Metadata filtering native. Mature ecosystem.
Qdrant	Good alt	Excellent vector search but separate infrastructure
Chroma	Prototype only	Not production-grade, no persistence
FAISS	Library, not DB	No metadata, no persistence — use as fallback only

Retrieval Order (Critical)

WRONG: Query → Embedding → Semantic Search → Rerank (labour law chunks can appear for cybercrime queries)
RIGHT: Query → Embedding → **Metadata filter (domain=X) FIRST** → Semantic Search → Cross-Encoder Rerank → Top 3

```
SELECT *, embedding <=> $1 AS distance
FROM legal_chunks
WHERE domain = $2           -- filter FIRST
ORDER BY distance
LIMIT 20;                  -- rerank these 20 down to 3
```

Ingestion Pipeline

Source	PDFs of IPC, CPC, Consumer Protection Act 2019, IT Act 2000, PWDVA etc. Stored in S3 or local /data/
Chunking	Section-aware chunking (NOT naive sliding window). Split on section boundaries. 512 tokens, 50-token overlap.
Metadata	{domain, act_name, section_number, subsection, source_url, chunk_id} stored per chunk.
Embedding	text-embedding-3-small via OpenAI API. Each chunk → 1536-dim vector.
Storage	INSERT into legal_chunks(chunk_id, text, embedding, metadata JSONB).
Re-ingestion	When a law changes, DELETE WHERE act_name=\$1 AND section=\$2, then re-embed only affected sections.

6. User & Conversation Storage

Database Schema

	<code>google_sub TEXT UNIQUE, email, name, created_at</code>	<i>Stable Google ID — never changes even if email does.</i>
	<code>user_id FK, domain, title (auto-generated), created_at, updated_at, extracted_facts JSONB, history_summary TEXT</code>	<i>One row per legal topic/session. Facts and summary live here, not in messages.</i>
	<code>conversation_id FK, role (user assistant), content, metadata JSONB {domain, plan_type, sources}, is_valid BOOL, created_at</code>	<i>Invalid queries: is_valid=false, but actually — don't write them at all. Check validity before INSERT.</i>

7. Conversation Memory & Window Management

LLMs are stateless. Every API call is a fresh instance. You reconstruct context manually each turn from the DB. Sending full history = unbounded cost + context window overflow + attention degradation on long contexts.

The Problem Without Windowing

Turn 1 costs 200 tokens. Turn 10 costs 2,000 tokens (all history). Turn 50 costs 10,000 tokens. You hit the context window limit and the model starts ignoring early messages even when technically present.

Three-Layer Memory Architecture

Layer 1 Sliding Window	How: Last 6 messages (3 turns) always included raw. Why: Simple, cheap. Loses facts mentioned >3 turns ago.
Layer 2 Extracted Facts	How: After every user message, a cheap LLM pass extracts legal entities (employer name, date of incident, amount, location) into <code>extracted_facts</code> JSONB on the conversations row. Why: Permanent across entire session. Independent of window size.
Layer 3 Rolling Summary	How: Every 10 messages, oldest 8 are compressed into a 3-4 sentence summary stored in <code>history_summary</code> . Raw old messages excluded from context. Why: Keeps old context without token cost.

What Each LLM Call Actually Receives

Section	Content	~Tokens
System Prompt	Domain instructions, output format, legal disclaimer	~200
History Summary	Compressed older turns (Layer 3) — only when exists	~150
Extracted Facts	Structured JSON of legal entities from entire session	~100
Last 6 Messages	Raw recent conversation (Layer 1)	~600
Current Query	User's new message this turn	~50

Retrieved Chunks	Top-3 reranked legal text chunks for this query	~800
TOTAL	Bounded, regardless of conversation length	~1900

8. Authentication Flow

- 1 User clicks 'Login with Google'**
React redirects to Google OAuth 2.0 Authorization Code URL with scopes: openid, email, profile
- 2 Google redirects to /auth/callback**
FastAPI endpoint receives ?code=... Exchanges code for tokens using authlib.
- 3 FastAPI creates JWT**
Signs JWT with HS256 + secret key. Payload: {user_id, email, exp}. Does NOT store Google token.
- 4 Set HttpOnly cookie**
response.set_cookie('session', jwt, httponly=True, secure=True, samesite='strict'). JS can never read this.
- 5 Every protected route**
FastAPI Depends(get_current_user) decodes JWT, verifies signature + expiry, returns user object.
- 6 Token refresh**
JWT expires in 24h. Refresh token (stored server-side in Redis) issues new JWT silently.

9. Critical Fixes Checklist

- Add session_id, user_id, clarification_count, error_state to LangGraph state
- 3-tier validity gate — never use LLM alone for legal yes/no
- Domain metadata filter BEFORE semantic search in all PGVector queries
- Define users / conversations / messages Postgres schema before building
- Give Planner explicit intent signals — not a free-form black box
- Section-aware chunking with act/section metadata per chunk
- Implement 3-layer memory: sliding window + facts extraction + rolling summary
- JWT in HttpOnly + Secure + SameSite=Strict cookie — never localStorage
- clarification_count hard cap at 2 — prevents infinite clarification loops
- Invalid queries: reject before DB write — nothing persisted